

Mini Project: Building a Robust Part-of-Speech Tagger

Natural Language Processing

Project Overview

In Natural Language Processing (NLP), resolving syntactic ambiguity is a core challenge. For instance, the word “watch” can be a noun (“my watch is broken”) or a verb (“watch the dog”). In this project, you will build a Hidden Markov Model (HMM) from scratch to assign the correct Part-of-Speech (POS) tag to each word in a sentence.

You will train and evaluate your model using the **Brown Corpus**. Moving from textbook theory to real-world data introduces two major challenges that you must solve programmatically:

1. **Out-of-Vocabulary (OOV) Words:** You must implement Laplace smoothing to handle words the model has never seen.
2. **Computational Underflow:** You must rewrite the Viterbi decoding algorithm in log-space to prevent floating-point collapse when processing long sentences.

Deliverables:

- Your well-commented Python source code (or Jupyter Notebook).
- A formal Written Report (4–6 pages).
- A Slide Presentation (7–10 slides) summarizing your findings.

Part 1: The Coding Implementation

Step 1: Data Acquisition and Splitting

Use the Natural Language Toolkit (`nltk`) to download the Brown Corpus using the `universal` tagset.

```
1 import nltk
2 nltk.download('brown')
3 nltk.download('universal_tagset')
4 from nltk.corpus import brown
5
6 # TODO: Load the corpus and split it into 80% training data and 20%
7 testing data.
```

Step 2: Parameter Estimation & Laplace Smoothing

Train your HMM by estimating the initial (π), transition (A), and emission (B) probabilities using Maximum Likelihood Estimation (MLE) based on frequency counts in your training set.

To prevent zero-probabilities when encountering unseen words during testing, apply **Laplace (Add- α) Smoothing** to your emission matrix. The smoothed probability of emitting word o_t given tag s_i is:

$$P_{\text{smoothed}}(o_t|s_i) = \frac{\text{Count}(o_t \text{ tagged as } s_i) + \alpha}{\text{Count}(s_i) + \alpha|V|} \quad (1)$$

Where $\alpha = 1.0$ and $|V|$ is the total number of unique words in your training vocabulary. Ensure you create a specific dictionary key (e.g., "<OOV>") to store the probability mass for unseen words.

Step 3: The Log-Space Viterbi Decoder

Because standard probabilities are strictly between 0 and 1, multiplying them together for a normal-length sentence results in infinitesimally small numbers (Underflow). Transform the recursive Viterbi step into log-space:

$$\log v_t(j) = \max_{i=1}^N \left[\log v_{t-1}(i) + \log P(s_j|s_i) \right] + \log P(o_t|s_j) \quad (2)$$

Hint: $\log(0)$ is mathematically undefined. Add a tiny constant (epsilon, 10^{-12}) to all probabilities before taking the logarithm to avoid math domain errors.

```
1 def log_viterbi(sentence, tags, A, B, pi):
2     # TODO: Initialize dynamic programming tables
3     # TODO: Perform the forward pass in log-space
4     # TODO: Backtrack to find the optimal sequence of tags
5     pass
6
```

Step 4: Evaluation

Write an evaluation loop to process the 20% test sentences and compute your total word-level accuracy.

$$\text{Accuracy} = \frac{\text{Total Correctly Tagged Words}}{\text{Total Words in Test Set}} \quad (3)$$

Part 2: The Written Report

Your report should be formatted as a formal academic paper (e.g., using an IEEE or ACL two-column format). It must include the following sections:

1. **Introduction:** Define the POS tagging problem and introduce Hidden Markov Models.
2. **Methodology:**
 - Explain how the data was partitioned.
 - Detail the parameter estimation process (MLE).
 - Clearly explain the mathematical necessity of Laplace Smoothing and the Log-Space transformation for the Viterbi algorithm.
3. **Results:** Present your final word-level accuracy on the 20% test set. Use tables or charts if you experimented with different α values for smoothing.

4. **Error Analysis:** (*Crucial*) Extract and present at least 3 to 5 specific sentences where your model failed. Analyze *why* it failed. Discuss how the “First-Order Markov Assumption” limits the model’s ability to handle long-distance grammatical agreements.
5. **Conclusion:** Summarize the effectiveness and limitations of HMMs for NLP.

Part 3: The Slide Presentation

You will deliver a 10-minute presentation to the class. Your slide deck (7–10 slides) should cover:

- **Title Slide**
- **The Problem:** What is POS tagging and why is ambiguity difficult?
- **The HMM Approach:** Briefly explain states, transitions, and emissions.
- **Engineering Hurdles:** Explain how you solved the Out-of-Vocabulary (OOV) and computational underflow problems.
- **Results & Evaluation:** What was your accuracy?
- **Error Case Study:** Walk the audience through one specific sentence where the model failed, explaining the limits of the Markov Assumption.
- **Q&A**

Grading Rubric

Category	Criteria	Points
Code Implementation	Correct estimation of A , B , and π . Proper implementation of Laplace smoothing and Log-Space Viterbi. Code runs without errors.	40
Written Report	Clear explanations of methodology. Strong, insightful error analysis identifying the theoretical limits of HMMs. Proper academic formatting.	35
Presentation	Clear verbal communication, professional slide design, ability to explain complex concepts (like log-space) simply, and handling of Q&A.	25
Total		100